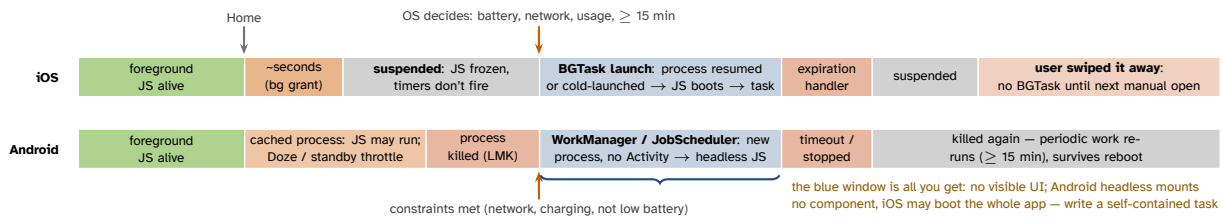


In one line: RN has **no background scheduler of its own**: JS runs only while the native process is alive and not suspended. Every background feature is a **native scheduler** — BGTaskScheduler, WorkManager, a push, a background mode — that wakes the app and **boots or resumes the JS runtime** for a short, OS-chosen window. The OS picks *when*; your code only picks *what*.



The options

- **expo-background-task** (SDK 53; expo-background-fetch is deprecated): Android **WorkManager**, iOS **BGTaskScheduler** (UIBackgroundModes: processing, id com.expo.modules.backgroundtask.processing). minimumInterval in **minutes**, default 12 h, floor 15. Needs battery + network; stops if the user kills the app; iOS: physical device only.
- **react-native-background-fetch** (Transistorsoft): iOS BGAppRefreshTask (com.transistorsoft.fetch), ≥ 15 min, frequency learnt from usage; Android stopOnTerminate: false + enableHeadless + startOnBoot; scheduleTask for one-off/periodic custom tasks.
- **Headless JS** (Android only): AppRegistry.registerHeadlessTask('Sync', () => task); a HeadlessJsTaskService returns HeadlessJsTaskConfig(name, data, 5000 /* ms timeout */, false) — the last flag: by default the app **crashes** if the task runs while in foreground. Retries: LinearCountingRetryPolicy(3, 1000) + throw HeadlessJsTaskError. From a BroadcastReceiver: acquireWakeLockNow().
- **Your own native job**: a Kotlin Worker / a BGTaskScheduler handler that does the work **natively** (upload, DB write) without booting JS — cheapest, most reliable; emit an event if JS happens to be alive. iOS: register every id **before the end of** didFinishLaunching, list it in BGTaskSchedulerPermittedIdentifiers; a second registration of the same id kills the app.
- **Push-triggered**: FCM data message → messaging().setBackgroundMessageHandler(fn) registered in index.js (outside components; on iOS it may mount the root); iOS silent push (content-available: 1) is low priority, throttled, and discarded for a force-quit app.
- **Continuous modes**: location (expo-location startLocationUpdatesAsync + TaskManager; Android foreground service + notification), audio (UIBackgroundModes: audio), BLE — held only while really in use. Big transfers: background URLSession / WorkManager, native side. iOS 26 BGContinuedProcessingTask: user-started work that continues with a system progress UI.

Testing

- iOS device, paused in LLDB after submit: e -l objc -- (void)[[BGTaskScheduler sharedScheduler] _simulateLaunchForTaskWithIdentifier:@"id"] (and _simulateExpiration..) — private, never ship it.
- Expo: triggerTaskWorkerForTestingAsync() (dev). Android: adb shell dumpsys jobscheduler (constraints met?); Doze: adb shell dumpsys deviceidle force-idle.

Example — expo-background-task

```
import * as TaskManager from 'expo-task-manager';
import * as BackgroundTask from 'expo-background-task';
const SYNC = 'outbox-sync';
// top-level scope: a headless launch mounts no component
TaskManager.defineTask(SYNC, async () => { // read storage, POST, write
  try { await flushOutbox(); // read storage, POST, write
    return BackgroundTask.BackgroundTaskResult.Success; }
  catch { return BackgroundTask.BackgroundTaskResult.Failed; }
});
export async function enableSync() {
  const s = await BackgroundTask.getStatusAsync();
  if (s !== BackgroundTask.BackgroundTaskStatus.Available)
    await BackgroundTask.registerTaskAsync(SYNC, { minimumInterval: 15 });
}
```

Writing a JS background task

- **Self-contained**: re-create what it needs (API client, tokens from Keychain, DB); do not assume the store, navigation or a logged-in React tree exists.
- **Short and idempotent**: an outbox flush, a delta sync, a prefetch. It can be cut off at any moment and run twice.
- Cold boot costs: the JS runtime + bundle load eats part of the window — keep the task's imports small; heavy work belongs in native.
- Keychain reads while locked need AFTER_FIRST_UNLOCK accessibility.

Platform limits

iOS	no guaranteed schedule; system learns usage; refresh tasks are brief, processing tasks longer (often when charging/idle) (<i>unverified</i>); nothing after a user force-quit; no BGTask in the simulator; silent push ≤ 2-3/hour, 30 s
Android	WorkManager periodic ≥ 15 min (flex ≥ 5 min); Doze + App Standby buckets defer work; constraints must all hold; OEM task killers (<i>unverified</i>); since 8.0 a background app can't startService() (IllegalStateException)

Traps

- ! “Every 15 minutes” — a *minimum*; iOS may run it hours later, or never.
- ! setInterval in JS as a background timer: frozen on suspend.
- ! Task defined inside a component — never registered in a headless launch.
- ! Headless JS on iOS — it doesn't exist.

Remember

The OS schedules, native wakes, JS gets a short window — design for “maybe, later”.